SCSM2223 CROSS-PLATFORM APPLICATION DEVELOPMENT

MOCK FINAL EXAMINATION (SET B) — SEMESTER 2, 2025/2026

ANSWER SCHEME AND EXAMINER NOTES

For self-marking after attempting Set B under exam conditions (3 hours, closed book).

SECTION A — MULTIPLE-CHOICE KEY (20 × 1 MARK)

No.	Ans	Why
1	**B**	Application tier = web server running the business logic.
2	**A**	MPA: server generates a complete new HTML page per request.
3	**D**	Hybrid apps run the web code inside a WebView container.
4	**C**	border-box: padding + border are included in the declared width.
5	**A**	auto-fit collapses empty tracks so items stretch (auto-fill leaves gaps).
6	**C**	confirm() returns true (OK) / false (Cancel). prompt() returns string/null.
7	**B**	Every render would attach ANOTHER listener — duplicate firing.
8	**D**	AJAX = background communication without a full page reload.
9	**C**	403 Forbidden: authenticated but not allowed. (401 = not authenticated.)
10	**A**	$(document).ready runs code only after the DOM is fully loaded.
11	**B**	Promises/.then chaining (and async/await) flatten callback hell.
12	**D**	.finally() always runs — fulfilled or rejected.
13	**C**	computed is cached; re-evaluates only when a dependency changes.
14	**B**	.prevent = event.preventDefault() — stops the page reload on submit.
15	**A**	<router-view> = placeholder where the matched route component renders.
16	**D**	mount() must come LAST, after app.use(router/pinia).
17	**B**	PATCH = partial update. PUT replaces the whole resource (missing fields wiped).
18	**A**	PSR-7 immutability: withHeader/withStatus return a NEW object — return it.
19	**D**	ERRMODE_EXCEPTION → PDO throws PDOException on database errors.
20	**C**	AuthN = “Who are you?”; AuthZ = “What can you do?”

⚠ ***Trap to watch:*** *Q9 is the 401-vs-403 trap in reverse (valid token, wrong owner → 403). Q17 is the PUT-vs-PATCH trap: PUT with only one field would WIPE the others. Q5 comes straight from the lab hint on auto-fit vs auto-fill.*

SECTION B — MODEL ANSWERS

QUESTION 1 (8 marks) — MPA vs SPA

a) MPA (server-centric): each user action sends an HTTP request to the server [1]; the server runs the logic and generates a COMPLETE new HTML page, and the browser reloads the entire page — rendering happens on the server [1]. SPA (client-centric): the page loads once; a user action triggers an AJAX/REST call [1]; the server returns only DATA (JSON via an API) and JavaScript in the browser re-renders just the affected section — rendering happens on the client, with no full reload [1].

b) Any two: content-heavy website; SEO-focused site; simple CRUD system; low frontend complexity [1+1].

c) Because in an SPA the UI is fully rendered on the client (component-based) and the backend is reached only through REST API calls [1], the same HTML/CSS/JS bundle can be wrapped in a native WebView container (Capacitor) and still function — it carries its own rendering and talks to the same API from inside the app [1].

QUESTION 2 (10 marks) — Asynchronous JavaScript

a) Callback hell is deeply nested callbacks (each async result needed by the next call), producing pyramid-shaped code that is hard to read, maintain, and error-handle [1.5]. Promises solve it by representing a future value that can be chained flat with .then() — each step returns a new Promise — with a single .catch() for errors (and async/await builds on this to read like synchronous code) [1.5].

b) The THREE errors [1 each]:

1. The function uses await but is not declared `async` — must be `async function getBook()`.

2. `response.json()` returns a Promise — it must be awaited: `const book = await response.json();`.

3. The catch block uses `error.message` but never receives the parameter — must be `catch (error) {`.

Corrected function [2]:

```
async function getBook() {
  try {
    const response = await fetch("https://api.example.com/books/7");
    if (!response.ok) {
      throw new Error(`HTTP Error! Status: ${response.status}`);
    }
    const book = await response.json();
    console.log(book.title);
  } catch (error) {
    console.log("Error:", error.message);
  }
}
```

⚠ ***Trap to watch:*** *Exactly the three most common real-exam code-writing losses: async keyword, await on .json(), catch parameter. If you missed any of the three here, drill this until automatic.*

c) Any two: XHR is verbose/callback-based, fetch is clean/Promise-based [1]; XHR error handling is messy, fetch has structured .catch/ok checking [1]; XHR is the old 1999 API, fetch is the modern ES6 standard (also supports streaming).

QUESTION 3 (12 marks) — Vue.js

a) Methods — regular functions called on demand (e.g., a click handler that adds an item) [1]. Computed — derived value, cached, re-runs only when its reactive dependencies change (e.g., doneCount from a todos array) [1]. Watchers — run side effects when a reactive value changes (e.g., logging or fetching whenever a username changes) [1].

b) ref() wraps ANY value (primitive or object) and exposes it through .value [1]; reactive() wraps only objects and is accessed directly without .value [1]. Rule of thumb taught: “use ref() by default — it works for everything” [1].

c) <router-link to="..."> renders an anchor that navigates with client-side routing (no page reload, auto ‘router-link-active’ class) [1]. <router-view /> is the placeholder where the component matched by the current route is rendered [1]. A dynamic segment /todos/:id captures a parameter; the component reads it with useRoute() → route.params.id [1].

d) The problem is prop drilling — passing the same prop through many intermediate layers that do not use it [1]. A Pinia store is a central reactive object any component can import and use directly, removing the need to relay props [1]. A store contains state (reactive data), getters (computed values), and actions (functions that change state) [1].

⚠ ***Trap to watch:*** *Set A tested SFC blocks and :key; Set B tests the OTHER half of the Vue tips list (computed/watchers, ref vs reactive rule, Router, Pinia). Between both sets you have covered the full Vue mastery checklist.*

QUESTION 4 (15 marks) — Backend, Security and Deployment

a) i) GET /products [1]. ii) PUT /products/12 [1]. iii) DELETE /products/12 [1]. (Nouns not verbs; plural resources; the verb carries the action.)

b) Header — the algorithm (alg, e.g., HS256) and type (typ: JWT) [1]. Payload — the claims: registered (iss, sub, exp, iat) and custom (e.g., role) [1]. Signature — HMAC-SHA256 over base64url(header) + “.” + base64url(payload) using the server’s secret key [1]. The payload is only base64url-ENCODED, not encrypted, so anyone can read it; but any edit changes the bytes, so the signature no longer matches when the server recomputes it — tampering is detected because the attacker does not know the secret key [1].

c) Stored XSS: the malicious script is SAVED in the database (e.g., in a comment) and served to every user who views it [1]. Reflected XSS: the script arrives in a URL or form value and is bounced straight back in the response without escaping [1]. Prevention — PHP side: escape on OUTPUT with htmlspecialchars($v, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8') [1]; Vue side: render with {{ }} (auto-escaped) and avoid v-html for user content [1].

d) Any two differences [1+1]: API base URL (localhost:8000 vs https production URL via VITE_API_BASE_URL); CORS (wide open * in dev vs strict allow-list in prod); TLS (optional vs REQUIRED + HSTS); build (Vite dev server/HMR vs npm run build static dist/); secrets (dev JWT secret must never be reused in prod). Vite inlines VITE_ variables into the bundle AT BUILD TIME [1]; the Capacitor app is just that built bundle inside a WebView, so if it was built pointing at localhost, the phone app would try to call localhost on the phone itself and fail — it must be built with the deployed API URL before npx cap sync [1].

SECTION C — MODEL CODE ANSWERS

QUESTION 5 (10 marks) — HTML Forms, Flexbox, DOM

a) Model answer:

```
<form action="#" method="post">
  <fieldset>
    <legend>Student Details</legend>
    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>
    <button type="submit">Register</button>
  </fieldset>
</form>
```

Marks: fieldset + legend [1]; label linked via for/id [1]; type="email" with required + submit button [1]. (The outer <form> tag is accepted but not required by the wording.)

b) Model answer:

```
.main-nav {
  display: flex;
  justify-content: space-between;
  position: sticky;
  top: 0;
}
```

Marks: display:flex [1]; justify-content: space-between [1]; position: sticky with top: 0 [1]. (Adding z-index is a bonus, not required.)

c) Model answer:

```
const list = document.getElementById("notesList");

list.addEventListener("click", function (e) {
  const btn = e.target.closest(".del");
  if (!btn) return;
  const ok = confirm("Are you sure?");
  if (!ok) return;
  btn.parentElement.remove();
});
```

Marks: single listener on the <ul> [1]; identifying the clicked button (closest/matches/className check) [1]; confirm() with early return on cancel [1]; removing the parent <li> via remove() (or removeChild) [1].

⚠ ***Trap to watch:*** *Attaching listeners to each .del button scores at most 1/4 — the question explicitly demands delegation, which is also why it keeps working for notes added later (bubbling).*

QUESTION 6 (10 marks) — FeedbackBoard.vue

Model answer:

```
<template>
  <div>
    <h2>{{ boardTitle }}</h2>
    <input v-model="comment" placeholder="Write a comment">
```

```
    <button @click="postComment">Post</button>
    <p>{{ total }} comments</p>
    <ul>
      <li v-for="c in comments" :key="c.id">
        {{ c.text }}
        <button @click="removeComment(c.id)">Remove</button>
      </li>
    </ul>
  </div>
</template>

<script setup>
import { ref, computed } from 'vue'

defineProps({
  boardTitle: { type: String, required: true }
})

const comment = ref('')
const comments = ref([])

const total = computed(() => comments.value.length)

function postComment() {
  const text = comment.value.trim()
  if (text === '') return
  comments.value.push({ id: Date.now(), text })
  comment.value = ''
}

function removeComment(id) {
  comments.value = comments.value.filter(c => c.id !== id)
}
</script>
```

Marks: SFC + <script setup> structure [1]; defineProps required String + {{ boardTitle }} [2]; ref comment + v-model [1]; post function with trim/guard/push object/clear [2]; v-for with :key="c.id" (id, NOT index — items are deletable!) [2]; remove by id (filter or splice with findIndex) [1]; computed total displayed [1].

⚠ ***Trap to watch:*** *This set deliberately makes items DELETABLE — so :key must be the id, not the index. Using the index here loses the :key mark (the exact anti-pattern the slides warn about). Also: .value everywhere in script, never in template.*

QUESTION 7 (15 marks) — Slim 4, PDO and JWT

a) Model answer:

```
$app->get('/api/books/{id}', function (Request $req, Response $res, array $args)
{
    $id = (int)$args['id'];
    $book = findBook($id);

    if ($book === null) {
        $res->getBody()->write(json_encode(['error' => 'Book not found']));
        return $res
            ->withHeader('Content-Type', 'application/json')
            ->withStatus(404);
    }
```

```
    $res->getBody()->write(json_encode($book));
    return $res
        ->withHeader('Content-Type', 'application/json')
        ->withStatus(200);
});
```

Marks: route pattern {id} + third $args parameter [1]; (int)$args['id'] cast [1]; null check returning 404 with error JSON [1]; success 200 with json_encode($book) [1]; withHeader + RETURN of the chained response in both branches [1].

⚠ ***Trap to watch:*** *Set A tested the POST/400/201 branch; the real paper can pick either verb. The immutability rule is identical: no return = no marks for that branch. withStatus(200) may be omitted (200 is default) — allowed.*

b) Model answer:

```
$sql = 'UPDATE books
        SET title = :title, author = :author, year = :year
        WHERE id = :id';
$stmt = $pdo->prepare($sql);
$stmt->execute([
    ':title'  => $title,
    ':author' => $author,
    ':year'   => $year,
    ':id'     => $id,
]);
$updated = $stmt->rowCount() > 0;
```

Marks: UPDATE ... SET with named placeholders [2]; WHERE id = :id included [1]; prepare + execute with value array [1]; rowCount() > 0 into $updated [1].

⚠ ***Trap to watch:*** *Forgetting the WHERE clause is catastrophic (updates EVERY row) — the slides joke about it for DELETE; same logic here. Each named placeholder must appear exactly once (HY093 gotcha).*

c) Model answer:

```
if (password_verify($password, $row['password_hash'])) {
    $payload = [
        'sub' => $row['id'],
        'exp' => time() + 3600,
    ];
    $jwt = JWT::encode($payload, $secret, 'HS256');
}
```

Marks: password_verify($password, $hash) — correct argument order [1]; payload with sub and exp = time()+3600 [1]; JWT::encode($payload, $secret, 'HS256') [1].

⚠ ***Trap to watch:*** *password_verify(plain, hash) — plain FIRST. Never compare hashes with == and never re-hash the submitted password to compare strings.*

d) Editing the payload changes its base64url bytes, so when the server recomputes the HMAC-SHA256 signature with its secret key during JWT::decode(), the recomputed signature no longer matches the token's signature — decode throws (SignatureInvalidException) [1]. The middleware rejects the request with 401 Unauthorized [1].

SET A + SET B — COMBINED COVERAGE MAP

Same blueprint, different samples: both sets follow the Dr's 20/45/35 cognitive split and the same topic weights (backend ≈ 33–34 marks, Vue ≈ 26, async ≈ 13, the rest fast fundamentals). Set B is deliberately one notch harder: spot-and-fix code (Q2b), rewrite-bad-URIs (Q4a), 404-branch route (Q7a), and deletable-list :key discipline (Q6).

Together the two sets cover the ENTIRE tips checklist: page-load cycle + roles (A-Q1) and MPA/SPA/hybrid (B-Q1); .then → async/await conversion (A) and error-fixing (B); semantic page + media query + pure-DOM (A) and forms + flexbox + delegation + BOM confirm (B); SFC/:key/ref-model-for-props component (A) and computed/emit-style remove/id-keys component (B); Slim POST+PDO INSERT+SQLi fix+Bearer header (A) and Slim GET/404+PDO UPDATE+password_verify+JWT::encode+tamper detection (B); REST concept (A) vs REST naming rewrites (B); JWT flow (A) vs JWT structure/tamper-proof (B); threats overview (A) vs XSS types (B); Capacitor concept (A) vs deployment env + prod-URL build rule (B).

Training protocol: sit Set B fresh (3 hours, closed book, handwritten). Mark with this scheme. Any code answer below full marks: retype it at the keyboard until it runs. Then re-sit whichever set scored lower, one day before the real paper — not the night before.